

SPM: C++ Essentials

Academic year 2025-2026

Prof. Massimo Torquati
massimo.torquati@unipi.it

Outline

- C++ Essentials for SPM (we assume C++20 or later)
 - Type deduction and the `auto` keyword
 - References and value semantics
 - The Standard Template Library (STL) model
 - Iterators
 - Standard Algorithms
 - Functors, lambdas, and the `std::function`
 - RAI objects
 - Move semantics
 - Rules of Five and Rule of Zero
 - Perfect Forwarding

Modern C++

- Performance and abstraction coexist
- Code is built around types
- Values and objects have lifetimes
- Resources are managed through objects
- Generic code is pervasive
- The Standard Template Library is central
- C++ aims at zero-overhead abstractions

Automatic type deduction (auto)

- `auto` reduces verbosity. It is not dynamic typing, the type is fixed at compile time.
- The compiler inspects the right-hand side (RHS) of the assignment and deduces the type from initialization.

Useful for:

- Complex types, e.g., iterator types from STL containers or types from lambda expressions
- To make code cleaner and more maintainable
- To work with templates and type traits
- The `const` qualifier and the reference operator (`&`) may change the type deduction process
 - `auto` drops top-level `const` and references, unless you write `auto&` or `const auto&`

```
auto x = 10;           // deduced as int
auto d = 3.14;         // deduced as double
auto str = "Hello";    // deduced as const char*

const int c = 1024;    // just a constant number
auto k = c;            // deduced as int (not const int)
const auto y = c;      // deduced as const int
auto& r = c;           // deduced as const int&

auto list={1,2,3};     // deduced as std::initializer_list<int>
                        // i.e., a read-only view of a list of values
std::vector<int> v{1,2,3};
auto it=v.begin();     // deduced as std::vector<int>::iterator
```

- `auto` is especially useful when the type is obvious from the RHS, or when the explicit type is verbose

decltype

- `decltype(expr)` returns the type of the expression without evaluating it
- It is useful in generic code, especially when the type depends on an expression
- Unlike `auto`, it preserves `const` and reference qualifiers

```
const int c = 1024;    // just a constant number
const int& r = c;      // const reference to c

auto a = r;            // deduced as int
decltype(r) b = c;     // deduced as const int&
decltype(c) x = c;     // deduced as const int
decltype((c)) y = c;   // deduced as const int& because
                       // (c) is an lvalue expression
```

```
// returns the type deduced from the expression a+b
template <typename T, typename U>
auto sum(T a, U b) -> decltype(a + b) {
    return a + b;
}
```


Automatic type deduction (cont.)

- Template argument deduction
 - The compiler determines the types from the arguments used in a function or class template instantiation
- In functions and lambdas

```
auto add(int a, int b) {  
    return a+b; // deduced as int  
}  
// generic lambda, it works for any type as long as the  
// types passed support the '*' operator  
auto F = [](auto a, auto b) {  
    return a * b;  
};
```

```
template <typename T, typename U>  
auto multiply(T x, U y) {  
    return x * y; // return type deduction  
}  
multiply(2, 3); // int return type  
multiply(2, 3.14); // double return type  
multiply(2, 3.14f); // float return type
```

Usual arithmetic conversions: operands are converted to a common type before evaluation.

- Promote small integer types to `int`
- If any operand is floating, convert to the highest floating rank present, `long double > double > float`
- Otherwise, for integers, choose the result type by rank and signedness. Mixed signed/unsigned can convert the result to unsigned

Examples:

- `2 + 3.5f` → float
- `'a' + 1` → int
- `(-1 < 1u)` // gives false, because -1 is converted to unsigned

List initialization `{}` vs Parentheses `()`

- `=`, `()`, and `{}` are not interchangeable
- Brace initialization prevents narrowing
- Key differences:
 - `{}` prefers `std::initializer_list` when available
 - `{}` prevents narrowing conversions
 - `{}` initializes aggregates, `()` does not
 - `{}` avoids “the most vexing parse”, i.e., avoids ambiguities where `()` may be interpreted as a function declaration
- Use `()` when you need a non-`initializer_list` overload

```
std::vector<int> a(3, 1); // vector of size 3 with all 1
std::vector<int> b{3, 1}; // vector with elements {3, 1}
```

```
int x{3.14};           // error, narrowing prevented
int y(3.14);           // ok, y == 3
```

// aggregate

```
struct Point { int x, y; };
Point p1{1, 2};        // ok
Point p2(1, 2);        // error
```

```
std::vector<int> v1{}; // empty vector
std::vector<int> v2(); // function declaration, not an object
```

```
std::string s1(3, 'a'); // "aaa"
std::string s2{'a', 'b'}; // "ab"
```


References in C++

- A reference is an alias to an existing object
 - A pointer stores the address of an object
- References enable aliasing without copying data
- `const&` is the standard read-only interface for large objects
- Value semantics vs reference semantics
 - many C++ objects behave like values
 - copying duplicates state
 - mutation is local unless aliases are introduced
 - value semantics simplifies reasoning and composition

```
int a = 5;
int& r = a;
r = 10;    // a becomes 10

void increment(int& x) { ++x; }

void print_sum(const std::vector<int>& v) {
    int s = 0;
    for (int x : v) s += x;
    std::cout << s << "\n";
}

std::vector<int> a{1,2,3};
auto b = a;    // copy
b[0] = 10;    // a is unchanged
```


The STL model

- Containers, accessed through iterators, and processed by algorithms

- containers store data
- iterators identify positions and ranges
- algorithms operate on iterator ranges
- this separation enables generic and reusable code

```
std::vector<int> v{1,2,3};  
v.push_back(4);  
std::sort(v.begin(), v.end());
```

- `std::vector` as the default sequence container

- contiguous storage
- cache-friendly access
- amortized `push_back`
- random access in $O(1)$
- usually the first container to consider
- In performance-oriented code, `std::vector` is often the baseline container.

Some useful containers:

- `std::vector`: dynamic contiguous array
- `std::array`: fixed-size array, stack allocated
- `std::list`: linked list
- `std::deque`: double-ended queue, insert $O(1)$
- `std::set`/`std::map`: ordered associative container (Red-Black Tree)
- `std::unordered_map`: hash table, average lookup $O(1)$

Iterators

- An iterator (it) is a **generalized pointer** that denotes a position in a sequence
- Core operations: dereference `*it`, advance `++it/--it`, and compare `it != last`, (some support `it += k`, `it[n]`, `<`)
- Standard Library algorithms operate on half-open ranges (i.e., `[first, last)`), and they are **container-agnostic**

```
std::vector<int> v(10, 1);    // v={1,1,1,1,1,1,1,1,1,1}
v[0]=10;
v[1]=9;                      // v={10,9,1,1,1,1,1,1,1,1}
auto first = v.begin() + 2;  // first points to index 2
auto last = v.end() - 3;     // last points to index 7 (excluded)
// fill in elements in the half-open range [first, last) starting from 0
std::iota(first, last, 0);    // v={10,9,0,1,2,3,4,1,1,1}
```


Iterators

- Types of iterators:
 - Output iterators - single pass, write
 - Input iterators - single pass, read
 - Forward iterators - multi-pass, forward only
 - Bidirectional iterators - multi-pass, forward and backward
 - Random-access iterators - constant time jumps (e.g., $it + n$, $it - n$)
 - Contiguous (C++20) - random-access plus contiguous memory layout
- Iterator categories form a hierarchy, stronger iterators support all operations of weaker ones
- An algorithm can be used with any iterator category that satisfies its requirement

```
std::istream_iterator<int>    // input
std::ostream_iterator<int>    // output
std::forward_list<int>::iterator // forward
std::list<int>::iterator       // bidirectional
std::vector<int>::iterator     // random-access, contiguous
std::span<int>::iterator       // contiguous
```

```
std::sort(v.begin(), v.end());    // vector OK
// std::sort(lst.begin(), lst.end()); // list: not allowed
lst.sort();                       // list-specific sort
```

Common pitfalls with Iterators

- Iterator invalidation

- Let's consider `std::vector`, it has a `size()` (i.e., the elements actually present) and a `capacity()` (i.e., the memory space allocated)
- When we `push_back` an element and `size() < capacity()` the element is inserted in-place, if there is no capacity the container reallocates space and the iterator may be no longer valid
- Erasing elements invalidates all iterators from that point on

- Dereferencing of `end()`

```
auto last = *v.end();    // WRONG!  
auto last = *(v.end() - 1); // Okay, or better v.back()
```

```
std::vector<int> v;  
v.reserve(10);    // set capacity =10; size is 0  
for (int i=0;i<9;++i) v.push_back(i);  
auto it = v.begin();  
v.push_back(10);  // size=10; capacity=10, 'it' still valid  
v.push_back(11);  // size=11; capacity=10 -> reallocation  
                  // 'it' may not be valid anymore
```

```
std::vector<int> v = {1,2,3,4,5};  
for (auto it=v.begin(); it != v.end(); ++it)  
    if (*it % 2 == 0) v.erase(it); // WRONG! ++it is not valid
```

```
std::vector<int> v = {1,2,3,4,5};  
for (auto it=v.begin(); it != v.end(); )  
    if (*it % 2 == 0) it = v.erase(it);  
    else ++it;
```

Example: iterators.cpp

Building a counting iterator

- Is it possible to build an iterator without a data container?
- Yes, we show how to build an iterator from a start value, computing each value as start + i (**no storage**)
 - The implementation
 - «yields by value», i.e., dereferencing returns a value, not a reference (there is nothing to refer to)
 - Models it as random-access (provide ++/--, +/-, [], distance, comparisons)
 - See counting_iterator.hpp (and its usage example counting_iterator.cpp)
- Why is it useful?
 - Zero allocation and no bandwidth spent on index arrays
 - Vectorization/parallelization friendly
- Limitation: not an output iterator (cannot write through it)

```
auto it = counting_iterator<std::size_t>(0);  
auto last = it + N;  
while (it != last) std::cout << *it++ << " ";
```

Example: counting_iterator.cpp

Standard Algorithms, mini intro

- Generic functions that work on iterator ranges [first, last)
- They separate **what** you do (the algorithm) from **where** the data is stored (e.g., containers)
- Many algorithms return iterators
- Common categories:
 - Non-modifying (e.g., `all_of`, `find_if`, `count`, ...)
 - Modifying (e.g., `transform`, `fill`, `iota`, ...)
 - Removal/compaction (e.g., `remove`, `unique`, ...)
 - Reordering (e.g., `sort`, `partition`, ...)
 - Numeric (e.g., `accumulate`, `reduce`, `inclusive_scan`, ...)
- Algorithms require specific iterator capabilities (e.g., `std::sort` needs Random Access Iterators)
- Many algorithms take predicates or callable objects, often written as lambdas

```
std::vector<int> v={ 1, -2, 3, -2, 5 };  
auto it=std::find(v.begin(), v.end(), 7); // not found, it == v.end()  
auto sum= std::accumulate(v.begin(), v.end(), 0); // sum=5  
std::vector<int> out(v.size()); // out same size of v  
std::transform(v.begin(), v.end(), out.begin(), std::negate<int>());  
// out={-1, 2, -3, 2, -5}  
// remove-erase idiom  
out.erase(std::remove(out.begin(), out.end(), 2), out.end());  
// out={-1, -3, -5}
```

Examples in algorithms.cpp

Function objects (functors)

- A functor is an object that defines `operator()`
- Unlike a plain function, a functor **can store arbitrary state**
- Functors are widely used in generic programming and STL algorithms
- Calls to `operator()` are often easy for the compiler to inline

// how to use MaxTracker (example)

```
std::vector<float> V = {1.0, 2.0, 3.0, 4.0};
```

```
MaxTracker<float> tracker(V[0]);  
std::for_each(V.begin(), V.end(), std::ref(tracker));  
std::cout << "Max= " << tracker.curr_max << "\n";
```

```
template <typename T>  
struct MaxTracker {  
    explicit MaxTracker(const T& v) : curr_max(v) {}  
  
    bool operator()(const T& v) {  
        if (v > curr_max) { curr_max=v; return true; }  
        return false;  
    }  
    T curr_max;  
};
```

`std::ref` here is used to avoid copying the functor.
It passes the functor by reference, so its internal state is updated in the original object rather than in the copy

Lambdas

- Lambdas are anonymous function objects that can be defined inline
- General form: `[capture](parameters) mutable → return_type { body }`
- `[capture]` specifies which external variables are used, and whether they are captured by value or by reference
 - `[&]` captures all variables **by reference**; `[=]` captures all variables **by value**
 - `init_capture`: initialization of variables in the capture list, can move or rename objects (e.g., `[p=std::move(ptr)]`)
- **mutable** (optional), allows modification of variables captured by value
- The **return_type** is optional because it is often deduced automatically

```
std::vector<int> v = {1, 2, 3, 4, 5};  
int factor = 2;  
  
std::for_each(v.begin(), v.end(),  
              [factor](int& x) { x *= factor; });
```

- Lambdas are the most convenient way to pass short custom behavior to STL algorithms

std::function

- Lambdas and functors have concrete compile-time types

`std::function< return-type (param1, param2, ...) >`

 - Different lambdas have different concrete types, even if they have the same signature. It is difficult to store lambdas in a single container.
- `std::function` provides type erasure (*) for callable objects
 - `std::function` can store: function pointers, lambdas, functors, `std::bind` expression, member function pointer, `std::packaged_task` objects
- More flexible, but may introduce extra overhead
 - `std::function` may allocate heap memory to store the captured variables
 - More boilerplate due to type erasure. If type erasure is unnecessary, prefer template functions or `auto` instead of `std::function`
 - Lambda functions with the same number of parameters and the same return type can be held by `std::function`
- Useful for event handlers, threading APIs and asynchronous execution
- `std::function` is more flexible but may introduce overhead

```
void print(const std::string& s) { std::cout << s; }
std::vector<std::function<void (const std::string&)>> F={
    print,
    [](const std::string& s){ std::cout << s; }
};
```

(*) Type erasure lets us store different callable types behind the same interface.

Compile-time vs runtime polymorphism

- templates and lambdas provide compile-time polymorphism
- **virtual functions** provide runtime polymorphism
- compile-time polymorphism often enables inlining and better optimization
- runtime polymorphism is useful when behavior is chosen dynamically
- The choice is often a trade-off between performance and runtime flexibility
- With templates, code is specialized for concrete types. With virtual functions, different objects share a common runtime interface

override asks the compiler to verify that a virtual function is really overriding a base-class virtual function

```
template <typename F>
void apply(F f, int x) {
    std::cout << f(x) << "\n";
}

apply([](int x) { return x * 2; }, 5); // F is the lambda type
```

```
struct Base {
    virtual int eval(int x) = 0;
    virtual ~Base() = default;
};

struct Double : Base {
    int eval(int x) override { return 2 * x; }
};

std::unique_ptr<Base> p = std::make_unique<Double>();
std::cout << p->eval(5) << "\n";
```


RAII objects

- **RAII** stands for **Resource Acquisition Is Initialization**
- An object acquires a resource in its constructor and releases it in its destructor
- The lifetime of the object controls the lifetime of the resource
- This is the standard C++ way to manage memory, files, locks, and other resources
- RAII is the foundation of safe code

```
std::mutex m;  
  
void update() {  
    std::lock_guard<std::mutex> lock(m);  
  
    // critical section here  
  
} // mutex released automatically here
```

```
void f() {  
    std::vector<int> v(1000); // memory acquired here  
  
    // use v here  
  
} // memory automatically released here
```

Move semantics

- Move semantics improves performance and resource management, enabling resources to be transferred (moved) rather than copied
 - Moving objects only makes sense if the object type owns a resource such as `std::string` and `std::vector` (containers in general)
- `std::move` is a utility function that casts its argument of type T to a T&& (rvalue reference), producing an xvalue
 - `std::move` does not move anything. It transforms from lvalue to xvalue
- An xvalue is a value that denotes a specific object that is eligible to be moved
- An xvalue is obtained from `std::move(x)`, `static_cast<T&&>(x)`, or as a result of a function that returns T&&
- Standard containers (e.g., `std::vector`) support move semantics

```
std::string s1{"Hello"};  
std::string s2=std::move(s1);
```

```
std::vector<int> v1 = {1, 2, 3, 4};  
std::vector<int> v2 = std::move(v1); // move the internal buffer
```


Move semantics (cont.)

- Move operations typically require non-const T&&. It uses **move constructors** and **move assignment operators**
- Moving from a **const** object usually selects copy, not move
- **A named variable is always an lvalue, even if its type is T&&**
- Therefore, inside a function, `std::move` is often needed
- After moving, an object is in a **valid but unspecified state** (you should not rely on old content)

```
const std::string s3 = "Hello";  
std::string s4 = std::move(s3); // copied, not moved
```

```
auto make_buffer() { return Buffer{}; }  
Buffer b = make_buffer(); // constructed (copy elision)  
Buffer b2(std::move(b)); // move constructed  
Buffer b3;  
b3 = make_buffer(); // move assigned
```

```
auto set_string(std::string&& s) {  
    mystring = std::move(s);  
}  
set_string("Hello"); // move assigned
```

If `std::move(s)` is not used, `s` will be copied not moved because `s` is an lvalue in the function scope

Move semantics and the Rule of Five

- Move semantics requires **move constructors** and **move assignment operators**
- These functions can be: a) compiler-generated; b) user-defined; c) explicitly **= default** or **= delete**
- If a class directly owns a raw resource, copy and move operations must be designed consistently
- **Rule of Five:** If you define, delete, or need to customize one of these, you should consider all five:

1. **Copy constructor;**
2. **Copy assignment operator**
3. **Move constructor**
4. **Move assignment operator**
5. **Destructor**

noexcept tells the compiler that a function does not throw exception. C++ containers prefer move constructor to be marked **noexcept** to enable move semantics. If not used it may fall back to copying.

```
class Buffer {  
    std::size_t size = 0;  
    int* data = nullptr;  
public:  
    // constructor  
    explicit Buffer(std::size_t n) : size(n), data(new int[n]) {}  
    // destructor  
    ~Buffer() { delete[] data; }  
    // copy constructor (deep copy)  
    Buffer(const Buffer& other); // deep copy  
    // copy assignment operator (deep copy)  
    Buffer& operator=(const Buffer& other);  
    // move constructor (transfer pointer)  
    Buffer(Buffer&& other) noexcept;  
    // move assignment operator (transfer pointer)  
    Buffer& operator=(Buffer&& other) noexcept;  
};
```

Example: buffer_example.cpp

Rule of Zero

- **Rule of Zero:** if a class does not directly manage raw resources, it should not define custom special member functions. **Let member objects manage themselves**
- Rule of Zero works well when members are already safe value types or **RAII** types, e.g.: `std::string`, `std::vector`, `std::unique_ptr`, `std::array`
- Move semantics does not imply manual implementation
- Manual move operations are needed only when the default generated ones are not correct or not available.
- If members already provide correct ownership semantics, the compiler-generated special members are usually enough

```
class Person {  
    std::string name;  
    std::vector<double> grades;  
public:  
    Person(std::string n, std::vector<double> g)  
        : name(std::move(n)), grades(std::move(g)) {}  
};
```

No destructor, no copy constructor,
no move constructor, no assignment operators needed

Forwarding references

- In a function template, a parameter of type `T&&` is a forwarding reference if `T` is deduced
- A forwarding reference can bind to both lvalues and rvalues
- `X&&` with explicit `X` is just an rvalue reference, not a forwarding reference
- `auto&&` behaves similarly in generic deduction contexts

```
template <typename T>  
void f(T&& x); // forwarding reference  
  
std::string s = "hello";  
f(s);           // T deduced as std::string&, x binds to lvalue  
f(std::string("hi")); // T deduced as std::string, x binds to rvalue
```


std::forward and Perfect forwarding

- Inside a function, a named parameter is always an lvalue
- `std::forward<T>(x)` preserves the original value category
- This allows the function to call copy or move overloads correctly
- This is called **perfect forwarding**

`std::move` casts to rvalue,
`std::forward<T>` conditionally
preserves lvalue/rvalue based on T

```
std::vector<std::string> V;  
void insert(const std::string& s) {  
    V.push_back(s); // calls the copy constructor  
}  
void insert(std::string&& s) {  
    V.push_back(std::move(s)); // calls the move constructor  
}  
template <typename T>  
void insertValue(T&& value) { // forwarding reference  
    // perfect forwarding, what insert function will be called, depends on the type of 'value'  
    insert(std::forward<T>(value));  
}
```

Code examples

- The file [some-cpp-features.cpp](#) shows some features of modern C++ we have discussed
- See also [simple_vector.cpp](#), a minimal home-made vector used only to illustrate essential container internals.
 - Disclaimer: do not use this version, prefer `std::vector`

Suggested Readings

- CPP reference <https://en.cppreference.com/w/cpp>
- «C++ High Performance» second edition by B. Andrist, V. Sehr, Packt
 - <https://www.packtpub.com/en-us/product/c-high-performance-9781787120952>
 - Git repo: <https://github.com/PacktPublishing/Cpp-High-Performance-Second-Edition>
 - Chapter 2 of this book provided as additional material